

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Computer Architecture Lab

Assignment

BATTLESHIP

Advisor(s): Phạm Quốc Cường

Student(s): Nguyễn Phúc Vĩnh 2414001

HO CHI MINH CITY, JUNE 2026

Contents

1	Giới thiệu	1
1.1	Tổng quan về trò chơi Battleship	1
1.2	Mục tiêu của Assignment	1
1.3	Kỹ năng MIPS được áp dụng	1
1.4	Cấu trúc báo cáo	2
2	Phân tích yêu cầu	3
2.1	Tóm tắt yêu cầu đề bài	3
2.2	Ràng buộc hợp lệ khi đặt tàu	3
2.2.1	Kiểm tra phạm vi (Boundary Check)	3
2.2.2	Kiểm tra hình dạng (Shape Check)	3
2.2.3	Kiểm tra độ dài (Length Check)	4
2.2.4	Kiểm tra chồng lấn (Overlap Check)	4
2.3	Biểu diễn giá trị trên bàn cờ	4
2.4	Yêu cầu về báo cáo	4
3	Tổng quan thiết kế chương trình	5
3.1	Phân chia chương trình thành các phần chính	5
3.1.1	Phần 1: Giao diện người dùng và chọn chế độ chơi	5
3.1.2	Phần 2: Giai đoạn thiết lập (Setup Phase)	5
3.1.3	Phần 3: Giai đoạn chiến đấu và kiểm tra người thắng	6
3.2	Luồng chủ yếu của chương trình	6
3.3	Thành phần chủ yếu của chương trình	7
3.3.1	Thủ tục điều phối (main)	7
3.3.2	Thủ tục giao diện	7
3.3.3	Thủ tục thiết lập	8
3.3.4	Thủ tục chiến đấu	8
3.3.5	Thủ tục hỗ trợ	8
3.4	Quy ước dữ liệu	8
4	Biểu diễn dữ liệu	9
4.1	Ánh xạ bàn cờ 2D thành mảng 1D	9
4.1.1	Công thức chuyển đổi index	9
4.1.2	Công thức tính byte offset	9
4.2	Cấu trúc dữ liệu chính	10

4.2.1	Bàn cờ chính (Main Board) - boardP1, boardP2	10
4.2.2	Bản đồ bắn (Memory Board) - memP1, memP2	10
4.2.3	Bộ đệm input (inputBuffer)	10
4.2.4	Mẫu bàn cờ tự động (autoBoardTemplate)	10
4.2.5	Biến chế độ chơi (gameMode)	10
4.3	Biểu diễn trực quan trên giao diện	11
4.3.1	Bàn cờ của chính mình (Own Board)	11
4.3.2	Bản đồ bắn đối phương (Enemy Map)	11
4.4	Ví dụ biểu diễn dữ liệu	11
5	Thuật toán giai đoạn thiết lập	12
5.1	Tổng quan giai đoạn Setup	12
5.2	Luồng chính của setup_player	12
5.3	Chi tiết thủ tục read_and_place_ship	13
5.4	Bước 1: Đọc input	13
5.5	Bước 2: Tách 4 số nguyên	13
5.6	Bước 3: Kiểm tra phạm vi	14
5.7	Bước 4: Kiểm tra hình dạng	14
5.8	Bước 5: Tính độ dài	14
5.9	Bước 6: Kiểm tra độ dài	15
5.10	Bước 7: Kiểm tra chồng lấn	15
5.11	Bước 8: Đặt tàu lên bàn cờ	15
5.12	Thủ tục hỗ trợ: parse_four_ints	16
5.13	Xử lý Computer trong chế độ One Player	16
5.14	Kiểm tra hợp lệ toàn bộ	17
6	Thuật toán giai đoạn chiến đấu	17
6.1	Tổng quan giai đoạn Battle	17
6.2	Luồng vòng lặp chính (game_loop)	17
6.3	Chi tiết thủ tục player_attack	19
6.4	Bước 1: Hiển thị bàn cờ	19
6.5	Bước 2: Yêu cầu nhập tọa độ tấn công	19
6.6	Bước 3: Kiểm tra phạm vi	19
6.7	Bước 4: Chuyển đổi tọa độ thành index	20
6.8	Bước 5: Kiểm tra tấn công lặp lại	20
6.9	Bước 6: Kiểm tra trúng hay hụt	20

6.10	Bước 7: Kiểm tra kết thúc trò chơi	21
6.11	Thủ tục hỗ trợ: board_has_ship	21
6.12	Quá trình cập nhật bàn cờ	21
6.13	Tình huống 1: Bắt trúng	22
6.14	Tình huống 2: Bắt hụt	22
6.15	Tình huống 3: Bắt lặp lại	22
6.16	Lưu ý quan trọng	22
7	Chế độ một người và hành động máy tính	23
7.1	Chế độ One Player vs Computer	23
7.2	Thiết lập bàn cờ Computer	23
7.3	Mẫu bàn cờ tự động (autoBoardTemplate)	23
7.4	Thủ tục nạp bàn cờ Computer	24
7.5	Chiến lược tấn công Computer	24
7.6	Thủ tục computer_attack	24
7.7	Chi tiết hoạt động	25
7.8	Ví dụ minh họa	26
7.9	Giới hạn của chiến lược Computer	27
7.10	So sánh hai chế độ chơi	27
8	Phân tích và xác thực Input	27
8.1	Tổng quan về xử lý input	27
8.2	Thủ tục parse_four_ints	28
8.3	Mục tiêu	28
8.4	Thuật toán	28
8.5	Chi tiết hoạt động	29
8.6	Ví dụ	30
8.7	Thủ tục parse_two_ints	30
8.8	Xác thực phạm vi (Range Validation)	32
8.9	Khi đặt tàu	32
8.10	Khi tấn công	32
8.11	Xử lý input sai	32
8.12	Vòng lặp input-validate-retry	33
8.13	Giới hạn hiện tại	33
8.14	Best Practice	34

9	Tổ chức thủ tục và khái niệm MIPS	34
9.1	Kiến trúc chương trình theo thủ tục	34
9.2	Gọi và trả về thủ tục (Procedure Call and Return)	34
9.3	Lệnh jal (jump-and-link)	34
9.4	Lệnh jr \$ra (jump register)	35
9.5	Ví dụ đơn giản	35
9.6	Bảo toàn thanh ghi (Register Preservation)	35
9.7	Loại thanh ghi	35
9.8	Sử dụng Stack để bảo toàn thanh ghi	36
9.8.1	Đẩy thanh ghi vào stack (Prologue)	36
9.8.2	Khôi phục thanh ghi từ stack (Epilogue)	36
9.9	Ví dụ thủ tục lồng nhau	36
9.10	Các syscall được sử dụng	37
9.11	Ví dụ sử dụng syscalls	37
9.11.1	In chuỗi	37
9.11.2	In số nguyên	37
9.11.3	Đọc chuỗi	38
9.12	Branching và Conditional Logic	38
9.13	Ví dụ: Kiểm tra phạm vi	38
9.14	Arithmetic Instructions	39
9.15	Ví dụ: Tính $\text{index} = \text{row} \times 7 + \text{col}$	39
9.16	Memory Instructions	39
9.17	Ví dụ: Đọc/ghi ô trên bàn cờ	40
10	Thiết kế giao diện người dùng	40
10.1	Tổng quan về giao diện	40
10.2	Menu chính	41
10.3	Hiển thị menu	41
10.4	Xử lý lựa chọn	41
10.5	Hướng dẫn (Help)	41
10.6	Lựa chọn chế độ chơi	42
10.7	Hiển thị lựa chọn chế độ	42
10.8	Xử lý chế độ	42
10.9	Giai đoạn Setup	43
10.10	Hướng dẫn đặt tàu	43
10.11	Phản hồi sau khi đặt tàu	43

10.12	Hiển thị bàn cờ	43
10.13	Bàn cờ của chính mình	44
10.14	Bản đồ bắn đối phương	44
10.15	Dual-Board Display	45
10.16	Giai đoạn chiến đấu	45
10.17	Yêu cầu nhập tọa độ	45
10.18	Thông báo kết quả	45
10.19	Tạm dừng để đổi lượt (Pause for Swap)	46
10.20	Thông báo kết thúc game	46
10.21	Thông báo lỗi	47
10.22	Nguyên tắc thiết kế	47
11	Kế hoạch kiểm thử và các Test Case	47
11.1	Tổng quan về kiểm thử	47
11.2	Các loại kiểm thử	48
11.3	Kiểm thử giai đoạn Setup	48
11.4	Kiểm thử giai đoạn Battle	48
11.5	Kiểm thử xác thực input	48
11.6	Kiểm thử tổng hợp	48
11.7	Bảng Test Case	48
11.8	Chiến lược kiểm thử	52
11.9	Kiểm thử từ dưới lên (Bottom-up Testing)	52
11.10	Kiểm thử biên (Boundary Testing)	52
11.11	Kiểm thử tiêu cực (Negative Testing)	53
11.12	Tiêu chí vượt qua kiểm thử	53
11.13	Công cụ kiểm thử	53
11.14	Báo cáo lỗi (Bug Report Format)	54
12	Giới hạn và cải tiến có thể	54
12.1	Giới hạn hiện tại	54
12.2	Chiến lược tấn công Computer	54
12.3	Ghi lại lịch sử di chuyển (Move History)	55
12.4	Kích thước bàn cờ cố định	55
12.5	Xử lý input hạn chế	55
12.6	Giao diện text thuần	55
12.7	Cải tiến có thể	56

12.8	Computer AI nâng cao	56
12.9	Ghi lại lịch sử trò chơi	56
12.10	Bàn cờ cấu hình được	56
12.11	Xử lý input tăng cường	57
12.12	Giao diện cấp cao hơn	57
12.13	Tính năng mạng	57
12.14	Thống kê và leaderboard	57
12.15	Mức ưu tiên cải tiến	58
12.16	Lý do không triển khai tính năng nâng cao	58
13	Kết luận	58
13.1	Tóm tắt chương trình	58
13.2	Các tính năng được triển khai	59
13.3	Kỹ năng MIPS được luyện tập	59
13.4	Thách thức và giải pháp	60
13.5	Thách thức: Quản lý bộ nhớ phức tạp	60
13.6	Thách thức: Bảo toàn thanh ghi khi thủ tục gọi thủ tục	60
13.7	Thách thức: Xác thực input phức tạp	60
13.8	Thách thức: Tách biệt boardP1/boardP2 và memP1/memP2	61
13.9	Đóng góp cho việc học	61
13.10	Kết luận cuối cùng	61
13.11	Hướng phát triển trong tương lai	62

List of Figures

List of Tables

4.1	Ký hiệu trên bàn cờ của mình	11
4.2	Ký hiệu trên bản đồ bắn đối phương	11
7.1	So sánh hai chế độ chơi	27
9.1	Các syscalls phổ biến trong MARS	37
9.2	Các lệnh branch và jump	38
9.3	Các lệnh toán học	39
9.4	Các lệnh truy cập bộ nhớ	40
12.1	Mức ưu tiên cải tiến	58

Listings



1 Giới thiệu

1.1 Tổng quan về trò chơi Battleship

Battleship là một trò chơi chiến lược cổ điển giữa hai người chơi. Mỗi người chơi quản lý một bộ hạm đội trên một bàn cờ hình vuông và cố gắng phát hiện cũng như tiêu diệt hạm đội của đối thủ. Trò chơi yêu cầu người chơi suy luận, dự đoán và tính toán xác suất để đánh trúng mục tiêu với số lần bắn ít nhất.

1.2 Mục tiêu của Assignment

Assignment này nhằm mục đích:

1. Mô phỏng trò chơi Battleship trên bàn cờ 7×7 bằng ngôn ngữ lắp ráp MIPS trong môi trường MARS simulator.
2. Triển khai đầy đủ giai đoạn thiết lập hạm đội (Setup Phase) và giai đoạn chiến đấu (Battle Phase).
3. Xây dựng giao diện người dùng rõ ràng, dễ sử dụng với các tùy chọn chế độ chơi (một người vs máy tính, hoặc hai người chơi).
4. Áp dụng và luyện tập các kỹ năng lập trình MIPS cơ bản và nâng cao.

1.3 Kỹ năng MIPS được áp dụng

Chương trình này sử dụng các khái niệm và kỹ thuật MIPS quan trọng:

- **Arithmetic Instructions:** Phép cộng, trừ, nhân để tính toán index mảng, độ dài tàu, và các giá trị tọa độ.
- **Memory/Data Transfer:** Lệnh lw (load word) và sw (store word) để đọc/ghi dữ liệu từ bộ nhớ.
- **Branch và Jump:** Lệnh beq, bne, blt để kiểm tra điều kiện và điều khiển luồng chương trình.
- **Procedure Call:** Lệnh jal (jump-and-link) và jr \$ra để gọi và trả về từ các thủ tục.
- **Stack Usage:** Sử dụng stack để lưu trữ \$ra và các thanh ghi được bảo toàn (\$s-registers) khi thủ tục gọi các thủ tục khác.

- **String/Integer Syscalls:** Các dịch vụ hệ thống để in chuỗi, in số, đọc số, đọc chuỗi, và đọc ký tự.
- **Array Indexing:** Tính toán offset mảng 2D được ánh xạ thành mảng 1D.
- **Conditional Logic:** Xử lý logic phức tạp để kiểm tra hợp lệ, phát hiện trùng/hệt, và xác định người thắng.

1.4 Cấu trúc báo cáo

Báo cáo này được tổ chức theo cấu trúc sau:

1. Phân tích yêu cầu (Requirement Analysis)
2. Tổng quan thiết kế chương trình (Program Design Overview)
3. Biểu diễn dữ liệu (Data Representation)
4. Thuật toán giai đoạn thiết lập (Setup Phase Algorithm)
5. Thuật toán giai đoạn chiến đấu (Battle Phase Algorithm)
6. Hành động của máy tính trong chế độ một người (One Player Mode and Computer Behavior)
7. Phân tích và xác thực input (Input Parsing and Validation)
8. Tổ chức thủ tục và khái niệm MIPS (Procedure Organization and MIPS Concepts Used)
9. Thiết kế giao diện người dùng (User Interface Design)
10. Kế hoạch kiểm thử và các test case (Testing Plan and Sample Test Cases)
11. Giới hạn và cải tiến có thể (Limitations and Possible Improvements)
12. Kết luận



2 Phân tích yêu cầu

2.1 Tóm tắt yêu cầu đề bài

Bài assignment yêu cầu cài đặt trò chơi Battleship với những đặc điểm sau:

- **Kích thước bàn cờ:** 7 hàng x 7 cột (49 ô).
- **Đội tàu mỗi người chơi:**
 - 3 tàu dài 2 ô
 - 2 tàu dài 3 ô
 - 1 tàu dài 4 ô
 - Tổng cộng: 6 tàu, 19 ô bị chiếm
- **Input tọa độ tàu:** Mỗi tàu được nhập bằng 4 giá trị tọa độ (row_bow, column_bow, row_stern, column_stern).
- **Giai đoạn thiết lập (Setup Phase):** Mỗi người chơi lần lượt đặt đủ 6 tàu theo thứ tự yêu cầu.
- **Giai đoạn chiến đấu (Battle Phase):** Hai người chơi lần lượt nhập tọa độ tấn công đối phương.
- **Kết thúc game:** Trò chơi kết thúc khi toàn bộ tàu của một người chơi bị tiêu diệt.

2.2 Ràng buộc hợp lệ khi đặt tàu

Một tàu được coi là hợp lệ khi đáp ứng tất cả các điều kiện sau:

2.2.1 Kiểm tra phạm vi (Boundary Check)

Tất cả tọa độ (hàng và cột) của tàu phải nằm trong khoảng từ 0 đến 6 (bao gồm). Nếu bất kỳ tọa độ nào vượt ra ngoài biên này, tàu bị từ chối.

2.2.2 Kiểm tra hình dạng (Shape Check)

Tàu phải nằm hoàn toàn ngang (cùng hàng, cột khác nhau) hoặc hoàn toàn dọc (cùng cột, hàng khác nhau). Tàu không được để dưới dạng chéo hoặc hình chữ L.

2.2.3 Kiểm tra độ dài (Length Check)

Độ dài của tàu được tính bằng công thức: $\text{độ dài} = |\text{tọa độ}_1 - \text{tọa độ}_2| + 1$. Độ dài tính được phải khớp chính xác với độ dài yêu cầu của tàu đang đặt:

- Tàu 1, 2, 3: độ dài phải = 2
- Tàu 4, 5: độ dài phải = 3
- Tàu 6: độ dài phải = 4

2.2.4 Kiểm tra chồng lấn (Overlap Check)

Trước khi đặt tàu, chương trình phải kiểm tra tất cả các ô mà tàu sẽ chiếm giữ trên bàn cờ. Nếu bất kỳ ô nào đã chứa tàu (giá trị = 1), tàu mới bị từ chối. Điều này đảm bảo không có hai tàu nào chồng lấn lên nhau.

2.3 Biểu diễn giá trị trên bàn cờ

Trên bàn cờ chính của mỗi người chơi:

- **Giá trị 1:** Ô chứa phần thân tàu còn nguyên vẹn (chưa bị bắn trúng).
- **Giá trị 0:** Ô trống hoặc đã bị bắn trúng (phần tàu đã bị tiêu diệt).

Trên bàn đồ bắn của mỗi người chơi (memory board):

- **Giá trị 0:** Ô chưa được tấn công (không rõ là gì).
- **Giá trị 1:** Ô đã được bắn nhưng trượt (MISS).
- **Giá trị 2:** Ô đã được bắn và trúng (HIT).

2.4 Yêu cầu về báo cáo

Báo cáo phải tuân thủ những yêu cầu sau:

1. **Không chứa code MIPS nguyên văn:** Chỉ được mô tả bằng lời, bảng, sơ đồ luồng, hoặc pseudocode mức cao.
2. **Tập trung vào ý tưởng, thuật toán, tổ chức dữ liệu:** Giải thích cách chương trình hoạt động từ góc độ logic và thiết kế.



3. **Gắn liền với implementation cụ thể:** Không viết quá chung chung; phải liên hệ với code MIPS thực tế.
4. **Sử dụng bảng và minh họa:** Khi cần, dùng bảng test case, pseudocode, hoặc flowchart để minh họa.

3 Tổng quan thiết kế chương trình

3.1 Phân chia chương trình thành các phần chính

Chương trình Battleship được chia thành ba phần lớn sau:

3.1.1 Phần 1: Giao diện người dùng và chọn chế độ chơi

Phần này quản lý:

- Hiển thị menu chính với các tùy chọn: Play Game, Instructions, Quit.
- Xử lý lựa chọn người dùng từ menu.
- Hiển thị hướng dẫn cách chơi khi người dùng chọn Instructions.
- Cho phép người dùng chọn chế độ chơi: One Player (vs Computer) hoặc Two Players.

3.1.2 Phần 2: Giai đoạn thiết lập (Setup Phase)

Phần này xử lý:

- Xóa toàn bộ bàn cờ của người chơi, đặt tất cả ô về giá trị 0.
- Yêu cầu người chơi nhập tọa độ từng tàu theo thứ tự (3 tàu dài 2, 2 tàu dài 3, 1 tàu dài 4).
- Kiểm tra tính hợp lệ của mỗi tàu (phạm vi, hình dạng, độ dài, chồng lấn).
- Nếu hợp lệ, đặt tàu lên bàn cờ; nếu không, yêu cầu nhập lại.
- Trong chế độ One Player, tự động đặt tàu cho Computer từ một mẫu được định nghĩa sẵn.



3.1.3 Phần 3: Giai đoạn chiến đấu và kiểm tra người thắng

Phần này quản lý:

- Vòng lặp chính của trò chơi (game loop) trong đó hai người chơi hoặc người chơi vs Computer lần lượt tấn công.
- Trong mỗi lượt: hiển thị bàn cờ của mình và bản đồ bắn đối phương, yêu cầu nhập tọa độ tấn công.
- Kiểm tra tọa độ nhập vào, xác định trúng (HIT) hay trượt (MISS).
- Cập nhật trạng thái bàn cờ và bản đồ bắn tương ứng.
- Sau mỗi tấn công, kiểm tra xem bàn cờ đối phương còn tàu không. Nếu không, thông báo người thắng và kết thúc game.

3.2 Luồng chủ yếu của chương trình

Luồng tổng quát của chương trình được mô tả như sau:

START

```
|
+----> SHOW MAIN MENU
|
+----> Choose Play Game
|   |
|   +----> CHOOSE GAME MODE
|       |
|       +----> One Player Mode OR Two Players Mode
|           |
|           +----> CLEAR BOTH BOARDS
|               |
|               +----> SETUP PLAYER 1
|                   (Input 6 ships)
|                   |
|                   +----> SETUP PLAYER 2 / COMPUTER
|                       (Input 6 ships or Auto)
|                       |
```



```
|
|                                     +---> BATTLE LOOP
|                                     |
|                                     +--> Player 1 Attack
|                                     |   Check WIN
|                                     |
|                                     +--> Player 2/Computer Attack
|                                     |   Check WIN
|                                     |
|                                     +---> Repeat until WIN
|                                     |
|                                     +---> ANNOUNCE WINNER
|                                     |
+---> Choose Instructions                                     +---> Back to MENU
|   (Display help text)
|   |
+---> Choose Quit
      (Exit program)
|
END
```

3.3 Thành phần chủ yếu của chương trình

Chương trình được tổ chức thành các thủ tục (procedure) chính sau:

3.3.1 Thủ tục điều phối (main)

Kiểm soát toàn bộ luồng chương trình: khởi tạo, menu lập, chọn chế độ, gọi setup, gọi battle loop, và xử lý kết thúc game.

3.3.2 Thủ tục giao diện

- **show_menu**: Hiển thị menu chính.
- **show_help**: Hiển thị hướng dẫn cách chơi.
- **choose_game_mode**: Cho phép chọn chế độ chơi.
- **print_board**: In một bàn cờ đơn lẻ.

- **print_dual_boards:** In song song bàn cờ của người chơi và bản đồ bắn đối phương.
- **pause_for_swap:** Tạo điểm dừng để đổi lượt (hai người chơi).

3.3.3 Thủ tục thiết lập

- **clear_board:** Xóa toàn bộ bàn cờ.
- **setup_player:** Yêu cầu người chơi đặt 6 tàu.
- **read_and_place_ship:** Đọc tọa độ, kiểm tra hợp lệ, đặt tàu.
- **load_auto_board:** Tự động đặt tàu cho Computer.

3.3.4 Thủ tục chiến đấu

- **game_loop:** Vòng lặp chính, điều khiển lượt tấn công của hai người chơi.
- **player_attack:** Xử lý một lượt tấn công của người chơi.
- **computer_attack:** Xử lý một lượt tấn công tự động của Computer.
- **board_has_ship:** Kiểm tra xem bàn cờ còn tàu hay không.

3.3.5 Thủ tục hỗ trợ

- **parse_four_ints:** Tách 4 số nguyên từ chuỗi input.
- **parse_two_ints:** Tách 2 số nguyên từ chuỗi input.

3.4 Quy ước dữ liệu

Chương trình sử dụng các biến toàn cục chính sau:

- **boardP1, boardP2:** Lưu trạng thái tàu của mỗi người chơi (1 = tàu còn sống, 0 = trống/bị phá).
- **memP1, memP2:** Lưu trạng thái tấn công của mỗi người chơi (0 = chưa biết, 1 = miss, 2 = hit).
- **gameMode:** Chế độ chơi (1 = One Player, 2 = Two Players).
- **inputBuffer:** Bộ đệm đọc input từ người dùng.

4 Biểu diễn dữ liệu

4.1 Ánh xạ bàn cờ 2D thành mảng 1D

Bàn cờ Battleship có kích thước 7 hàng $\times$ 7 cột, tổng cộng 49 ô. Để đơn giản hóa xử lý trong MIPS, bàn cờ 2D được ánh xạ thành một mảng 1D với 49 phần tử.

4.1.1 Công thức chuyển đổi index

Khi người chơi cung cấp tọa độ (row, column) với row, column $\in [0, 6]$, ta tính index trong mảng 1D như sau:

$$\text{index} = \text{row} \times 7 + \text{column} \quad (4.1)$$

Ví dụ:

- $(0, 0) \rightarrow \text{index} = 0 \times 7 + 0 = 0$ (ô đầu tiên, góc trái trên)
- $(0, 6) \rightarrow \text{index} = 0 \times 7 + 6 = 6$ (ô cuối cùng hàng 0, góc phải trên)
- $(6, 6) \rightarrow \text{index} = 6 \times 7 + 6 = 48$ (ô cuối cùng, góc phải dưới)
- $(3, 3) \rightarrow \text{index} = 3 \times 7 + 3 = 24$ (ô giữa bàn cờ)

4.1.2 Công thức tính byte offset

Vì mỗi phần tử trong mảng là một word (4 byte), byte offset của phần tử tại index được tính:

$$\text{byte_offset} = \text{index} \times 4 \quad (4.2)$$

Hoặc kết hợp cả hai:

$$\text{byte_offset} = (\text{row} \times 7 + \text{column}) \times 4 \quad (4.3)$$

Trong MIPS, khi muốn đọc/ghi ô tại (row, column), ta:

1. Tính $\text{index} = \text{row} \times 7 + \text{column}$
2. Tính $\text{byte_offset} = \text{index} \times 4$
3. Dùng $t0 = \text{base_address} + \text{byte_offset}$ để truy cập phần tử
4. Dùng lw hoặc sw để đọc/ghi

4.2 Cấu trúc dữ liệu chính

4.2.1 Bàn cờ chính (Main Board) - boardP1, boardP2

Lưu trạng thái tàu của mỗi người chơi. Mỗi phần tử là một 32-bit word:

- **Giá trị 1:** Ô chứa phần thân tàu còn nguyên vẹn (chưa bị bắn trúng).
- **Giá trị 0:** Ô trống hoặc đã bị bắn trúng.

Kích thước: $49 \times 4 \text{ byte} = 196 \text{ byte}$ mỗi bàn.

4.2.2 Bản đồ bắn (Memory Board) - memP1, memP2

Lưu lịch sử tấn công của mỗi người chơi. Mỗi phần tử là một 32-bit word:

- **Giá trị 0:** Ô chưa được tấn công (unknown).
- **Giá trị 1:** Ô đã được tấn công nhưng trượt (MISS).
- **Giá trị 2:** Ô đã được tấn công và trúng (HIT).

Kích thước: $49 \times 4 \text{ byte} = 196 \text{ byte}$ mỗi bản đồ.

4.2.3 Bộ đệm input (inputBuffer)

Một mảng ký tự (bytes) để đọc chuỗi input từ người dùng. Kích thước đủ để chứa một dòng input (khoảng 100-256 byte).

4.2.4 Mẫu bàn cờ tự động (autoBoardTemplate)

Một bàn cờ được xác định sẵn cho Computer trong chế độ One Player. Chứa 49 phần tử với các ô tàu đã được đặt sẵn.

4.2.5 Biến chế độ chơi (gameMode)

Một word 32-bit lưu chế độ chơi:

- **Giá trị 1:** One Player vs Computer.
- **Giá trị 2:** Two Players.

4.3 Biểu diễn trực quan trên giao diện

Khi in bàn cờ, các ký hiệu được sử dụng để biểu diễn trạng thái ô:

4.3.1 Bàn cờ của chính mình (Own Board)

Giá trị	Ký hiệu	Ý nghĩa
1	[1]	Phần tàu còn sống
0 (trống)	.	Ô trống, chưa bị bắn
0 (bị phá)	X	Ô bị bắn trúng, tàu bị phá

Table 4.1: Ký hiệu trên bàn cờ của mình

Chú ý: Trên bàn cờ chính, giá trị 0 có thể biểu diễn cả ô trống và ô bị phá hủy. Để phân biệt, chương trình có thể sử dụng thêm logic hoặc sử dụng ký hiệu khác tùy thuộc vào cách in bàn cờ.

4.3.2 Bản đồ bắn đối phương (Enemy Map)

Giá trị	Ký hiệu	Ý nghĩa
0	.	Chưa biết (chưa bắn)
1	O	Đã bắn nhưng trượt (MISS)
2	X	Đã bắn và trúng (HIT)

Table 4.2: Ký hiệu trên bản đồ bắn đối phương

4.4 Ví dụ biểu diễn dữ liệu

Giả sử người chơi 1 đặt 2 tàu:

- Tàu 1 (dài 2): từ (0, 0) đến (0, 1) - nằm ngang
- Tàu 2 (dài 2): từ (2, 3) đến (3, 3) - nằm dọc

Bàn cờ boardP1 sẽ có giá trị 1 tại các vị trí:

- Index 0 (row=0, col=0) = 1

- Index 1 (row=0, col=1) = 1
- Index 17 (row=2, col=3) = 1
- Index 24 (row=3, col=3) = 1

Các vị trí khác = 0.

Sau khi bắn vào (0, 0) và trúng, boardP1 tại index 0 được đặt về 0, còn memP2 (bản đồ bắn của Player 2) tại index 0 được đặt về 2 (HIT).

5 Thuật toán giai đoạn thiết lập

5.1 Tổng quan giai đoạn Setup

Giai đoạn thiết lập là bước đầu tiên sau khi chọn chế độ chơi. Mục tiêu của giai đoạn này là cho phép mỗi người chơi đặt đúng 6 tàu (theo đặc tả: 3 tàu dài 2, 2 tàu dài 3, 1 tàu dài 4) lên bàn cờ của mình.

5.2 Luồng chính của setup_player

Thủ tục setup_player thực hiện theo luồng sau:

```
PROCEDURE setup_player(player_id)
|
+----> Clear board của player
        (Đặt tất cả 49 ô về 0)
|
+----> FOR i = 1 to 3 (Đặt 3 tàu dài 2)
|   |
|   +----> Display "Place ship of length 2"
|   |
|   +----> CALL read_and_place_ship(2)
|   |
|   +----> IF valid: continue to next ship
|   |       ELSE: repeat this ship
|
+----> FOR i = 1 to 2 (Đặt 2 tàu dài 3)
|   |
```

```
|      +---> Display "Place ship of length 3"
|      |
|      +---> CALL read_and_place_ship(3)
|      |
|      +---> IF valid: continue to next ship
|      |      ELSE: repeat this ship
|
+---> FOR i = 1 to 1 (Đặt 1 tàu dài 4)
|
|      +---> Display "Place ship of length 4"
|      |
|      +---> CALL read_and_place_ship(4)
|      |
|      +---> IF valid: setup complete
|      |      ELSE: repeat this ship
```

END PROCEDURE

5.3 Chi tiết thủ tục read_and_place_ship

Thủ tục read_and_place_ship là thủ tục chính xử lý việc đặt một tàu. Nó thực hiện các bước sau:

5.4 Bước 1: Đọc input

Đọc một dòng từ người dùng vào inputBuffer. Dòng này dự kiến chứa 4 số nguyên cách nhau bởi khoảng trắng: row_bow, column_bow, row_stern, column_stern.

```
READ input_line into inputBuffer
```

5.5 Bước 2: Tách 4 số nguyên

Gọi thủ tục parse_four_ints để tách 4 số từ chuỗi input. Nếu không thể tách được đúng 4 số, báo lỗi và trả về.

```
CALL parse_four_ints(inputBuffer, addr_r1, addr_c1, addr_r2, addr_c2)
```

```
IF parse failed:
```

```
    PRINT "Invalid input format. Please try again."
```



RETURN with failure

Sau bước này, ta có:

- $r_1 = \text{row_bow}$, $c_1 = \text{column_bow}$
- $r_2 = \text{row_stern}$, $c_2 = \text{column_stern}$

5.6 Bước 3: Kiểm tra phạm vi

Kiểm tra tất cả 4 tọa độ có nằm trong $[0, 6]$ hay không.

```
IF (r1 < 0 OR r1 > 6 OR c1 < 0 OR c1 > 6 OR  
    r2 < 0 OR r2 > 6 OR c2 < 0 OR c2 > 6):  
    PRINT "Coordinates out of range [0, 6]!"  
    RETURN with failure
```

5.7 Bước 4: Kiểm tra hình dạng

Kiểm tra tàu có nằm ngang hoặc dọc.

```
is_horizontal = (r1 == r2) AND (c1 != c2)  
is_vertical = (r1 != r2) AND (c1 == c2)  
  
IF NOT (is_horizontal OR is_vertical):  
    PRINT "Ship must be horizontal or vertical!"  
    RETURN with failure
```

5.8 Bước 5: Tính độ dài

Tính độ dài của tàu dựa vào tọa độ.

```
IF is_horizontal:  
    length = ABS(c2 - c1) + 1  
ELSE IF is_vertical:  
    length = ABS(r2 - r1) + 1
```

5.9 Bước 6: Kiểm tra độ dài

So sánh độ dài tính được với độ dài yêu cầu.

```
required_length = [parameter passed to function]
```

```
IF length != required_length:
```

```
    PRINT "Ship length does not match! Expected: " + required_length
```

```
    RETURN with failure
```

5.10 Bước 7: Kiểm tra chồng lấn

Quét tất cả ô mà tàu sẽ chiếm, kiểm tra không có ô nào đã chứa tàu (giá trị = 1).

```
FOR each cell (r, c) that ship occupies:
```

```
    index = r * 7 + c
```

```
    byte_offset = index * 4
```

```
    value = board[byte_offset]
```

```
    IF value != 0:
```

```
        PRINT "Ship overlaps with existing ship!"
```

```
        RETURN with failure
```

```
END FOR
```

5.11 Bước 8: Đặt tàu lên bàn cờ

Nếu tất cả kiểm tra đều thành công, ghi giá trị 1 vào tất cả ô mà tàu chiếm giữ.

```
FOR each cell (r, c) that ship occupies:
```

```
    index = r * 7 + c
```

```
    byte_offset = index * 4
```

```
    board[byte_offset] = 1
```

```
END FOR
```

```
PRINT "Ship placed successfully!"
```

```
RETURN with success
```


5.12 Thủ tục hỗ trợ: parse_four_ints

Thủ tục này tách 4 số nguyên từ một chuỗi input. Nó quét chuỗi từ trái sang phải, bỏ qua khoảng trắng, và thu thập các chữ số để xây dựng số.

```
PROCEDURE parse_four_ints(input_string, addr_val1, addr_val2,
                           addr_val3, addr_val4)

    count = 0
    current_number = 0

    FOR each character in input_string:
        IF character is a digit (0-9):
            current_number = current_number * 10 + (digit value)
        ELSE IF character is whitespace OR end of string:
            IF current_number has been accumulated:
                Store current_number in appropriate address (addr_val1, etc.)
                count += 1
                current_number = 0

            IF count == 4:
                RETURN success

    IF count != 4:
        RETURN failure

END PROCEDURE
```

5.13 Xử lý Computer trong chế độ One Player

Trong chế độ One Player, Computer không cần đặt tàu tương tác. Thay vào đó, chương trình sao chép một bàn cờ mẫu được định nghĩa sẵn (autoBoardTemplate) vào boardP2 của Computer.

```
PROCEDURE load_auto_board()
    FOR i = 0 to 48:
        boardP2[i] = autoBoardTemplate[i]
```

```
PRINT "Computer board loaded."
```

```
END PROCEDURE
```

Mẫu boardTemplate được xác định sẵn sao cho các tàu được đặt hợp lệ và không chồng lấn.

5.14 Kiểm tra hợp lệ toàn bộ

Sau khi setup hoàn thành cho cả hai người chơi, chương trình có thể thêm một hàm kiểm tra toàn bộ để đảm bảo:

1. Mỗi board có chính xác 19 ô chứa tàu ($1 + 1 + 1 + 1 + 1 + 2 + 2 + 2 + 3 + 3 + 4 = 19$).
2. Không có ô nào vượt ra ngoài biên.

Tuy nhiên, nếu các kiểm tra được thực hiện đúng từng bước trong `read_and_place_ship`, kiểm tra toàn bộ này là không cần thiết.

6 Thuật toán giai đoạn chiến đấu

6.1 Tổng quan giai đoạn Battle

Giai đoạn chiến đấu (Battle Phase) bắt đầu sau khi cả hai người chơi (hoặc người chơi và Computer) đã đặt xong tàu. Trong giai đoạn này, hai đối thủ lần lượt tấn công bàn cờ của nhau cho đến khi toàn bộ tàu của một người bị tiêu diệt.

6.2 Luồng vòng lặp chính (game_loop)

Vòng lặp chính điều khiển các lượt tấn công của từng người chơi:

```
PROCEDURE game_loop()
```

```
  LOOP WHILE TRUE:
```

```
    +---> PLAYER 1 TURN
```

```
    |      |
```

```
    |      +---> CALL player_attack(Player1, Player2, memP1, boardP2)
```

```
    |      |
```



```
|          +---> Display Player 1's board and memP1 (enemy map)
|          +---> Read target row, column
|          +---> Check if hit or miss
|          +---> Update boardP2 and memP1
|          +---> Return hit/miss result
|
|          +---> CALL board_has_ship(boardP2)
|          |
|          +---> IF no ship remains:
|                  PRINT "Player 1 WINS!"
|                  RETURN from game_loop
|
|          +---> IF Two Players mode:
|                  CALL pause_for_swap()
|                  (Display message and wait for confirmation)
|
+---> PLAYER 2 / COMPUTER TURN
|
|          +---> IF One Player mode:
|                  CALL computer_attack(Computer, Player1, memP2, boardP1)
|          |
|          ELSE IF Two Players mode:
|                  CALL player_attack(Player2, Player1, memP2, boardP1)
|          |
|          +---> Display Player 2's board and memP2 (enemy map)
|          +---> Read target row, column
|          +---> Check if hit or miss
|          +---> Update boardP1 and memP2
|          +---> Return hit/miss result
|
|          +---> CALL board_has_ship(boardP1)
|          |
|          +---> IF no ship remains:
|                  PRINT "Player 2 / Computer WINS!"
|                  RETURN from game_loop
|
```



```
|      +---> IF Two Players mode:  
|          CALL pause_for_swap()  
  
END LOOP
```

6.3 Chi tiết thủ tục `player_attack`

Thủ tục `player_attack` xử lý một lượt tấn công của người chơi. Nó thực hiện các bước sau:

6.4 Bước 1: Hiển thị bàn cờ

In bàn cờ của người chơi hiện tại và bản đồ bắn đối phương (enemy map).

```
CALL print_dual_boards(own_board, enemy_memory_board)
```

6.5 Bước 2: Yêu cầu nhập tọa độ tấn công

Yêu cầu người chơi nhập tọa độ hàng và cột mà người chơi muốn tấn công.

```
PRINT "Enter target row and column (e.g., '3 5'):"  
READ input_line  
CALL parse_two_ints(input_line, addr_row, addr_col)  
  
IF parse failed:  
    PRINT "Invalid input format. Please try again."  
    RETURN
```

6.6 Bước 3: Kiểm tra phạm vi

Kiểm tra tọa độ nhập vào có nằm trong $[0, 6]$ hay không.

```
IF (row < 0 OR row > 6 OR col < 0 OR col > 6):  
    PRINT "Coordinates out of range [0, 6]!"  
    RETURN
```

6.7 Bước 4: Chuyển đổi tọa độ thành index

Tính index từ tọa độ (row, column).

```
index = row * 7 + col  
byte_offset = index * 4
```

6.8 Bước 5: Kiểm tra tấn công lặp lại

Kiểm tra xem ô này đã được tấn công trước đó hay chưa bằng cách kiểm tra giá trị tương ứng trên bản đồ bắn (enemy_memory_board).

```
memory_value = enemy_memory_board[byte_offset]  
  
IF memory_value != 0: (memory_value là 1 hoặc 2)  
    PRINT "You already attacked this cell! Try another."  
    RETURN
```

6.9 Bước 6: Kiểm tra trúng hay hụt

Kiểm tra giá trị tại ô đó trên bàn cờ của đối thủ.

```
opponent_value = opponent_board[byte_offset]  
  
IF opponent_value == 1:  
    // HIT  
    PRINT "HIT!"  
  
    // Update opponent's board  
    opponent_board[byte_offset] = 0  
  
    // Update attacker's memory  
    enemy_memory_board[byte_offset] = 2  
  
ELSE: // opponent_value == 0  
    // MISS  
    PRINT "MISS!"
```



```
// Update attacker's memory  
enemy_memory_board[byte_offset] = 1
```

6.10 Bước 7: Kiểm tra kết thúc trò chơi

Sau mỗi lượt tấn công, kiểm tra xem bàn cờ của đối thủ còn tàu hay không.

```
CALL board_has_ship(opponent_board)
```

```
IF no ship remains:  
    PRINT "You won! Game over."  
    RETURN true (game over signal)  
ELSE:  
    RETURN false (game continues)
```

6.11 Thủ tục hỗ trợ: board_has_ship

Thủ tục board_has_ship kiểm tra xem bàn cờ còn bất kỳ ô nào chứa tàu (giá trị = 1) hay không.

```
PROCEDURE board_has_ship(board)  
    FOR i = 0 to 48:  
        IF board[i] == 1:  
            RETURN true (still has ship)  
  
    RETURN false (no ship remains)  
  
END PROCEDURE
```

Nếu không còn ô nào có giá trị 1, thì tất cả tàu của người chơi đó đã bị phá hủy, và trò chơi kết thúc.

6.12 Quá trình cập nhật bàn cờ

Để hiểu rõ hơn cơ chế cập nhật dữ liệu, ta xem xét một ví dụ:

6.13 Tình huống 1: Bắn trúng

Giả sử Player 1 bắn vào (2, 3):

1. $\text{index} = 2 \times 7 + 3 = 17$
2. $\text{byte_offset} = 17 \times 4 = 68$
3. Kiểm tra $\text{memP1}[17]$ (bản đồ bắn của Player 1):
 - Nếu $\text{memP1}[17] \neq 0$: ô đã bắn rồi, từ chối.
 - Nếu $\text{memP1}[17] == 0$: chưa bắn, tiếp tục.
4. Kiểm tra $\text{boardP2}[17]$ (bàn cờ thực của Player 2):
 - Nếu $\text{boardP2}[17] == 1$: trúng! Cập nhật $\text{boardP2}[17] = 0$, $\text{memP1}[17] = 2$.
 - Nếu $\text{boardP2}[17] == 0$: trượt. Cập nhật $\text{memP1}[17] = 1$.

6.14 Tình huống 2: Bắn hụt

Nếu ô (2, 3) trên boardP2 là 0 (trống):

1. Chỉ cập nhật $\text{memP1}[17] = 1$ (MISS).
2. $\text{boardP2}[17]$ vẫn là 0 (không thay đổi).

6.15 Tình huống 3: Bắn lặp lại

Nếu $\text{memP1}[17]$ đã là 1 hoặc 2 (đã bắn trước đó):

1. Chương trình từ chối tấn công, yêu cầu nhập tọa độ mới.
2. Không cập nhật bất kỳ dữ liệu nào.

6.16 Lưu ý quan trọng

- **Tách biệt $\text{boardP1}/\text{boardP2}$ và $\text{memP1}/\text{memP2}$:** Điều này rất quan trọng. Board chính chỉ được cập nhật khi đối thủ bắn trúng ô của mình. Memory board chỉ được cập nhật với kết quả tấn công của mình đối thủ.
- **Thứ tự kiểm tra:** Luôn kiểm tra memory board trước để đảm bảo không bắn lại cùng ô. Điều này giúp ngăn ngừa lỗi logic.



- **Hiển thị kết quả rõ ràng:** Sau mỗi lượt tấn công, in ra HIT! hoặc MISS! rõ ràng để người chơi biết kết quả.

7 Chế độ một người và hành động máy tính

7.1 Chế độ One Player vs Computer

Chế độ One Player cho phép một người chơi (Player 1) đấu với Computer (Player 2). Lợi thế của chế độ này là:

- Người chơi không cần người bạn để chơi.
- Computer tự động đặt tàu, không cần nhập liệu.
- Computer tự động tấn công, giảm khối lượng tương tác.

7.2 Thiết lập bàn cờ Computer

Trong chế độ One Player, Computer không cần trải qua giai đoạn setup tương tác. Thay vào đó, chương trình sao chép một mẫu bàn cờ được định nghĩa sẵn vào boardP2.

7.3 Mẫu bàn cờ tự động (autoBoardTemplate)

Mẫu autoBoardTemplate là một mảng 49 phần tử được xác định sẵn trong phần khai báo dữ liệu của chương trình. Mỗi phần tử có giá trị 0 hoặc 1, biểu diễn trạng thái ô trên bàn cờ Computer.

Ví dụ, một mẫu có thể sắp xếp tàu như sau:

```
0 1 2 3 4 5 6
0 [1] [1] [0] [0] [0] [0] [0]  <- Ship 1 (length 2, horizontal)
1 [0] [0] [0] [0] [0] [0] [0]
2 [1] [0] [0] [1] [1] [1] [0]  <- Ship 3 (length 3, horizontal)
3 [1] [0] [0] [0] [0] [0] [0]  <- Ship 2 (length 2, vertical)
4 [1] [0] [1] [0] [0] [0] [0]  <- Ship 4 continues (length 2 vertical)
5 [0] [0] [1] [0] [0] [0] [0]
6 [0] [0] [1] [1] [1] [1] [0]  <- Ship 5 (length 4, horizontal)
```


7.4 Thủ tục nạp bàn cờ Computer

Khi chọn chế độ One Player, sau khi Player 1 hoàn thành setup, chương trình gọi `load_auto_board()`:

```
PROCEDURE load_auto_board()
  FOR i = 0 to 48:
    byte_offset = i * 4
    boardP2[byte_offset] = autoBoardTemplate[i]

  PRINT "Computer board loaded successfully."

END PROCEDURE
```

Thủ tục này sao chép toàn bộ 49 phần tử từ mẫu vào `boardP2`. Mẫu được thiết kế sao cho:

1. Không có ô nào vượt ra ngoài biên.
2. Các tàu được đặt hợp lệ (ngang hoặc dọc).
3. Không có tàu nào chồng lấn.
4. Đúng số lượng ô tàu: 19 ô ($3 \times 2 + 2 \times 3 + 1 \times 4$).

7.5 Chiến lược tấn công Computer

Chiến lược tấn công của Computer trong phiên bản này là chiến lược tuyến tính (linear scanning strategy). Đây là chiến lược đơn giản, dễ kiểm thử, nhưng không phải AI nâng cao.

7.6 Thủ tục `computer_attack`

```
PROCEDURE computer_attack(Computer, Player1, memP2, boardP1)

  // Tìm ô chưa bắn đầu tiên theo thứ tự
  FOR index = 0 to 48:
    byte_offset = index * 4
    memory_value = memP2[byte_offset]
```

```
IF memory_value == 0:  (ô chưa bắn)

    // Tính tọa độ để hiển thị
    row = index / 7
    col = index % 7

    PRINT "Computer attacks: (" + row + ", " + col + ")"

    // Kiểm tra trúng hay hụt
    IF boardP1[byte_offset] == 1:
        PRINT "HIT!"
        boardP1[byte_offset] = 0
        memP2[byte_offset] = 2
    ELSE:
        PRINT "MISS!"
        memP2[byte_offset] = 1

    // Kiểm tra kết thúc game
    CALL board_has_ship(boardP1)
    IF no ship remains:
        PRINT "Computer wins!"
        RETURN game_over

    RETURN

END FOR

END PROCEDURE
```

7.7 Chi tiết hoạt động

Chiến lược này hoạt động như sau:

1. **Quét từ index 0 đến 48:** Computer duyệt qua tất cả 49 ô trên bàn cờ theo thứ tự từ trên xuống dưới, từ trái sang phải.
2. **Tìm ô chưa bắn:** Khi gặp ô đầu tiên có `memP2[index] == 0` (chưa bắn), Computer



chọn ô đó làm mục tiêu.

3. **Tấn công:** Computer bắn vào ô được chọn.

- Nếu $\text{boardP1}[\text{index}] == 1$: Trúng! Cập nhật $\text{boardP1}[\text{index}] = 0$, $\text{memP2}[\text{index}] = 2$.
- Nếu $\text{boardP1}[\text{index}] == 0$: Trượt! Cập nhật $\text{memP2}[\text{index}] = 1$.

4. **Kiểm tra kết thúc:** Sau mỗi tấn công, kiểm tra xem Player 1 còn tàu hay không.

5. **Trả về:** Thử tục kết thúc, quay lại game loop, đến lượt Player 1 tấn công.

7.8 Ví dụ minh họa

Giả sử memP2 (lịch sử tấn công của Computer) hiện tại là:

Indices 0-6: 0, 0, 1, 0, 0, 0, 0
Indices 7-13: 0, 1, 0, 0, 0, 0, 0
Indices 14-20: 0, 0, 0, 0, 0, 0, 0
...

Computer sẽ:

1. Quét từ index 0: $\text{memP2}[0] == 0$? Có. Chọn (0, 0).
2. Bắn vào (0, 0), giả sử $\text{boardP1}[0] == 0$ (trượt). Cập nhật $\text{memP2}[0] = 1$.
3. Kết thúc lượt, quay lại game loop.
4. Lần tiếp theo, Computer quét từ index 0 lại:
 - $\text{memP2}[0] == 1$? Có, bỏ qua.
 - $\text{memP2}[1] == 0$? Có. Chọn (0, 1).
5. Tiếp tục cho đến khi tất cả ô được tấn công hoặc game kết thúc.

7.9 Giới hạn của chiến lược Computer

Chiến lược tuyến tính này có các giới hạn:

- **Không thông minh:** Computer không học hỏi từ các lần trúng trước. Nếu bắn trúng ô (2, 3), nó vẫn không biết nên bắn vào ô lân cận để tìm tàu.
- **Không ngẫu nhiên:** Chiến lược là xác định (deterministic). Người chơi có thể dự đoán được ô Computer sẽ bắn tiếp theo.
- **Không tối ưu:** Có thể tốn nhiều lượt bắn hơn cần thiết để tiêu diệt tất cả tàu.

Những giới hạn này là chấp nhận được cho mục đích assignment này, vì mục tiêu là luyện tập lập trình MIPS, không phải phát triển AI chơi game.

7.10 So sánh hai chế độ chơi

Khía cạnh	One Player	Two Players
Setup Player 1	Nhập liệu	Nhập liệu
Setup Player 2	Tự động (autoBoardTemplate)	Nhập liệu
Attack Player 1	Nhập liệu	Nhập liệu
Attack Player 2	Tự động (computer_attack)	Nhập liệu
Pause for swap	Không cần	Có
Độ phức tạp	Thấp	Cao
Thích hợp với AI	Có	Không

Table 7.1: So sánh hai chế độ chơi

8 Phân tích và xác thực Input

8.1 Tổng quan về xử lý input

Chương trình cần xác thực input từ người dùng ở hai bối cảnh chính:

1. **Giai đoạn Setup:** Đọc 4 tọa độ khi đặt tàu.
2. **Giai đoạn Battle:** Đọc 2 tọa độ khi tấn công.

Xác thực input đảm bảo chương trình không bị lỗi khi người dùng nhập sai định dạng hoặc giá trị ngoài phạm vi.

8.2 Thủ tục parse_four_ints

Thủ tục này tách 4 số nguyên từ một chuỗi input. Nó được sử dụng khi đặt tàu.

8.3 Mục tiêu

Chuyển đổi một chuỗi nhập vào (ví dụ: “0 0 0 1”) thành 4 giá trị số nguyên.

8.4 Thuật toán

```
PROCEDURE parse_four_ints(input_string, addr_val1, addr_val2,  
                           addr_val3, addr_val4)
```

```
    parsed_count = 0  
    current_number = 0  
    current_char_index = 0  
    is_building_number = false
```

```
    WHILE current_char_index < length of input_string:  
        char = input_string[current_char_index]
```

```
        IF char is a digit (0-9):  
            // Add digit to current number  
            digit_value = char - '0'  
            current_number = current_number * 10 + digit_value  
            is_building_number = true
```

```
        ELSE IF char is whitespace (space, tab, newline):
```

```
            IF is_building_number:  
                // Number is complete, store it  
                SWITCH parsed_count:  
                    CASE 0: *addr_val1 = current_number  
                    CASE 1: *addr_val2 = current_number  
                    CASE 2: *addr_val3 = current_number  
                    CASE 3: *addr_val4 = current_number
```

```
            parsed_count += 1  
            current_number = 0
```

```
is_building_number = false

IF parsed_count == 4:
    RETURN success

ELSE:
    // Invalid character (not digit, not whitespace)
    // In some implementations, this could trigger an error
    // For simplicity, we skip invalid characters
    ;

current_char_index += 1

// Handle last number if exists
IF is_building_number AND parsed_count < 4:
    SWITCH parsed_count:
        CASE 0: *addr_val1 = current_number
        CASE 1: *addr_val2 = current_number
        CASE 2: *addr_val3 = current_number
        CASE 3: *addr_val4 = current_number
    parsed_count += 1

IF parsed_count == 4:
    RETURN success
ELSE:
    RETURN failure

END PROCEDURE
```

8.5 Chi tiết hoạt động

Thủ tục này hoạt động theo các bước sau:

1. **Khởi tạo:** Đặt `parsed_count = 0`, `current_number = 0`, và chỉ số vị trí ký tự `= 0`.
2. **Quét từng ký tự:** Lặp qua từng ký tự trong chuỗi input.

- Nếu ký tự là chữ số: Nhân `current_number` với 10 và cộng giá trị chữ số mới (để xây dựng số multi-digit).
- Nếu ký tự là khoảng trắng: Nếu đang xây dựng số, lưu số đó vào địa chỉ tương ứng, tăng `parsed_count`, và đặt lại `current_number`.
- Nếu ký tự khác: Bỏ qua (hoặc có thể báo lỗi).

3. **Xử lý số cuối cùng:** Sau vòng lặp, nếu vẫn đang xây dựng số, lưu nó.

4. **Kiểm tra:** Nếu `parsed_count == 4`, trả về thành công. Ngược lại, thất bại.

8.6 Ví dụ

Input: "2 3 2 4"

1. Quét '2': digit, `current_number = 2`.
2. Quét ' ': whitespace, lưu 2 vào `addr_val1`, `parsed_count = 1`, `current_number = 0`.
3. Quét '3': digit, `current_number = 3`.
4. Quét ' ': whitespace, lưu 3 vào `addr_val2`, `parsed_count = 2`, `current_number = 0`.
5. Quét '2': digit, `current_number = 2`.
6. Quét ' ': whitespace, lưu 2 vào `addr_val3`, `parsed_count = 3`, `current_number = 0`.
7. Quét '4': digit, `current_number = 4`.
8. End of string: lưu 4 vào `addr_val4`, `parsed_count = 4`.
9. Trả về success.

8.7 Thủ tục `parse_two_ints`

Thủ tục này tương tự `parse_four_ints` nhưng chỉ tách 2 số nguyên. Được sử dụng khi tấn công.

```
PROCEDURE parse_two_ints(input_string, addr_val1, addr_val2)
    // Tương tự parse_four_ints nhưng lặp đến khi parsed_count == 2

    parsed_count = 0
    current_number = 0
    current_char_index = 0
    is_building_number = false

    WHILE current_char_index < length of input_string:
        char = input_string[current_char_index]

        IF char is digit:
            digit_value = char - '0'
            current_number = current_number * 10 + digit_value
            is_building_number = true

        ELSE IF char is whitespace:
            IF is_building_number:
                SWITCH parsed_count:
                    CASE 0: *addr_val1 = current_number
                    CASE 1: *addr_val2 = current_number

                parsed_count += 1
                current_number = 0
                is_building_number = false

            IF parsed_count == 2:
                RETURN success

        current_char_index += 1

    IF is_building_number AND parsed_count < 2:
        SWITCH parsed_count:
            CASE 0: *addr_val1 = current_number
            CASE 1: *addr_val2 = current_number
```



```
parsed_count += 1
```

```
IF parsed_count == 2:  
    RETURN success  
ELSE:  
    RETURN failure
```

```
END PROCEDURE
```

8.8 Xác thực phạm vi (Range Validation)

Sau khi tách số nguyên từ chuỗi input, chương trình phải kiểm tra các giá trị có nằm trong phạm vi cho phép hay không.

8.9 Khi đặt tàu

```
// Kiểm tra 4 tọa độ có nằm trong [0, 6]  
IF (r1 < 0 OR r1 > 6 OR c1 < 0 OR c1 > 6 OR  
    r2 < 0 OR r2 > 6 OR c2 < 0 OR c2 > 6):  
    PRINT "Error: Coordinates must be in range [0, 6]!"  
    RETURN failure
```

8.10 Khi tấn công

```
// Kiểm tra 2 tọa độ có nằm trong [0, 6]  
IF (row < 0 OR row > 6 OR col < 0 OR col > 6):  
    PRINT "Error: Coordinates must be in range [0, 6]!"  
    RETURN failure
```

8.11 Xử lý input sai

Khi input không hợp lệ, chương trình:

1. **Thông báo lỗi rõ ràng:** In thông báo lỗi cụ thể (ví dụ: “Invalid format”, “Out of range”, v.v.).
2. **Yêu cầu nhập lại:** Quay lại bước đọc input, yêu cầu người dùng nhập lại.



3. **Không cập nhật trạng thái:** Không thay đổi bất kỳ dữ liệu nào trên bàn cờ hoặc bản đồ bản.

8.12 Vòng lặp input-validate-retry

LOOP:

```
PRINT "Enter coordinates: "  
READ input_line  
  
CALL parse_two_ints(input_line, addr_row, addr_col)  
  
IF parse failed:  
    PRINT "Invalid input format. Please try again."  
    CONTINUE LOOP  
  
IF row < 0 OR row > 6 OR col < 0 OR col > 6:  
    PRINT "Coordinates out of range [0, 6]. Please try again."  
    CONTINUE LOOP  
  
// Valid input, proceed  
BREAK LOOP
```

8.13 Giới hạn hiện tại

Chương trình xác thực input có những giới hạn sau:

- **Không xử lý số âm:** Input có số âm sẽ không được phân tích đúng. Ví dụ, “-1 2” sẽ không được tách thành -1 và 2.
- **Không xử lý số quá lớn:** Nếu input chứa số lớn hơn 32-bit integer, hành vi không xác định.
- **Không xử lý ký tự đặc biệt:** Input chứa ký tự đặc biệt (trừ khoảng trắng) sẽ bị bỏ qua hoặc gây lỗi.

Những giới hạn này là chấp nhận được vì input được giả định là từ người chơi hợp lý, không phải từ input tương tác phức tạp.

8.14 Best Practice

Khi viết code MIPS để xác thực input:

1. **Tách biệt parse và validate:** Parse (tách số) riêng, validate (kiểm tra phạm vi) riêng.
2. **Cung cấp feedback rõ ràng:** Thông báo lỗi phải chỉ ra chính xác vấn đề gì.
3. **Lặp lại cho đến hợp lệ:** Không bỏ qua input sai; yêu cầu người dùng nhập lại.
4. **Kiểm thử kỹ lưỡng:** Test với input hợp lệ, không hợp lệ, và biên (boundary).

9 Tổ chức thủ tục và khái niệm MIPS

9.1 Kiến trúc chương trình theo thủ tục

Chương trình Battleship được tổ chức thành nhiều thủ tục nhỏ, mỗi thủ tục chịu trách nhiệm một chức năng cụ thể. Cách tổ chức này mang lại các lợi ích:

- **Dễ đọc:** Mỗi thủ tục là một khối logic độc lập, dễ hiểu.
- **Tái sử dụng:** Thủ tục có thể được gọi nhiều lần từ các vị trí khác nhau.
- **Dễ bảo trì:** Khi cần sửa lỗi hoặc cải tiến, chỉ cần sửa thủ tục tương ứng.
- **Modular:** Các thủ tục có thể phát triển độc lập.

9.2 Gọi và trả về thủ tục (Procedure Call and Return)

Trong MIPS, việc gọi thủ tục sử dụng lệnh `jal` (jump-and-link) và trả về sử dụng `jr $ra`.

9.3 Lệnh `jal` (jump-and-link)

Lệnh này có chức năng:

1. Lưu địa chỉ lệnh kế tiếp ($PC + 4$) vào thanh ghi `$ra` (return address register).
2. Nhảy tới địa chỉ của thủ tục.

```
jal procedure_name    # Gọi thủ tục
```



Ví dụ:

```
jal print_board      # Gọi thủ tục print_board
                     # \($ra sẽ chứa địa chỉ lệnh kế tiếp
```

9.4 Lệnh jr \$ra (jump register)

Lệnh này trả về từ thủ tục bằng cách nhảy tới địa chỉ lưu trong \$ra.

```
jr $ra               # Trả về từ thủ tục
```

9.5 Ví dụ đơn giản

```
main:
    jal simple_proc    # Gọi simple_proc
    # Trở lại đây sau khi simple_proc kết thúc

simple_proc:
    # ... code ...
    jr $ra              # Trả về
```

9.6 Bảo toàn thanh ghi (Register Preservation)

Khi một thủ tục gọi thủ tục khác, cần bảo toàn giá trị của các thanh ghi quan trọng.

9.7 Loại thanh ghi

MIPS có hai loại thanh ghi cần xem xét khi gọi thủ tục:

1. **\$s-registers** (\$s0-\$s7): Thanh ghi lưu trữ tạm thời. Thủ tục được gọi có trách nhiệm bảo toàn những thanh ghi này trước khi sử dụng.
2. **\$t-registers** (\$t0-\$t9): Thanh ghi tạm thời. Thủ tục gọi không thể giả định những thanh ghi này vẫn giữ giá trị cũ sau khi gọi thủ tục khác.
3. **\$ra**: Thanh ghi return address. Nếu thủ tục gọi thủ tục khác (không chỉ gọi bổ sung), \$ra cần được bảo toàn.

9.8 Sử dụng Stack để bảo toàn thanh ghi

Khi thủ tục cần bảo toàn giá trị, nó sử dụng stack (vùng nhớ được quản lý bởi \$sp - stack pointer).

9.8.1 Đẩy thanh ghi vào stack (Prologue)

Tại đầu thủ tục, nếu thủ tục cần sử dụng \$s-registers hoặc gọi thủ tục khác:

```
procedure_name:
    # Prologue: save registers
    sw $ra, offset($sp) # Lưu \ $ra
    sw $s0, offset($sp) # Lưu \ $s0 nếu cần
    sw $s1, offset($sp) # Lưu \ $s1 nếu cần
    addi $sp, $sp, -12   # Điều chỉnh stack pointer (lưu 3 từ = 12 byte)
```

9.8.2 Khôi phục thanh ghi từ stack (Epilogue)

Trước khi trả về, khôi phục các thanh ghi:

```
# Epilogue: restore registers
addi $sp, $sp, 12   # Điều chỉnh stack pointer
lw $s1, offset($sp) # Khôi phục \ $s1
lw $s0, offset($sp) # Khôi phục \ $s0
lw $ra, offset($sp) # Khôi phục \ $ra
jr $ra              # Trả về
```

9.9 Ví dụ thủ tục lồng nhau

```
outer_proc:
    # Prologue
    sw $ra, 0($sp)
    addi $sp, $sp, -4

    # Code...
    jal inner_proc      # Gọi inner_proc
    # Trở lại

    # Epilogue
```

```
addi $sp, $sp, 4
lw $ra, 0($sp)
jr $ra
```

```
inner_proc:
# inner_proc không cần lưu $ra nếu không gọi thủ tục khác
# ... code ...
jr $ra
```

9.10 Các syscall được sử dụng

Chương trình sử dụng các dịch vụ hệ thống (syscalls) để tương tác với người dùng:

Code	Tên	Input	Mô tả
1	print_int	\$a0 = giá trị	In một số nguyên
2	print_float	\$f12 = giá trị	In một số thực
4	print_string	\$a0 = địa chỉ	In một chuỗi
5	read_int		Đọc một số nguyên, trả về trong \$v0
8	read_string	\$a0 = địa chỉ buffer, \$a1 = độ dài	Đọc một chuỗi
10	exit		Kết thúc chương trình
12	read_char		Đọc một ký tự, trả về trong \$v0

Table 9.1: Các syscalls phổ biến trong MARS

9.11 Ví dụ sử dụng syscalls

9.11.1 In chuỗi

```
la $a0, message      # Tải địa chỉ chuỗi
li $v0, 4             # Code 4: print_string
syscall
```

9.11.2 In số nguyên

```
li $a0, 42           # Giá trị cần in
li $v0, 1            # Code 1: print_int
syscall
```



9.11.3 Đọc chuỗi

```
la $a0, input_buffer # Địa chỉ buffer
li $a1, 100           # Độ dài tối đa
li $v0, 8             # Code 8: read_string
syscall
```

9.12 Branching và Conditional Logic

Chương trình sử dụng các lệnh branch để kiểm tra điều kiện:

Lệnh	Ý nghĩa	Ví dụ
beq	Branch if equal	beq \$t0, \$t1, label
bne	Branch if not equal	bne \$t0, \$t1, label
blt	Branch if less than	blt \$t0, \$t1, label
ble	Branch if less or equal	ble \$t0, \$t1, label
bgt	Branch if greater than	bgt \$t0, \$t1, label
bge	Branch if greater or equal	bge \$t0, \$t1, label
j	Jump	j label

Table 9.2: Các lệnh branch và jump

9.13 Ví dụ: Kiểm tra phạm vi

```
# Kiểm tra nếu $t0 < 0
blt $t0, $zero, out_of_range
```

```
# Kiểm tra nếu $t0 > 6
bgt $t0, 6, out_of_range
```

```
# Hợp lệ
# ... tiếp tục ...
j continue
```

```
out_of_range:
    # Xử lý lỗi
```

```
la $a0, error_msg
li $v0, 4
syscall
```

continue:

9.14 Arithmetic Instructions

Chương trình sử dụng các phép toán:

Lệnh	Mô tả
add \$d, \$s, \$t	$\$d = \$s + \$t$
addi \$d, \$s, imm	$\$d = \$s + \text{immediate}$
sub \$d, \$s, \$t	$\$d = \$s - \$t$
mul \$d, \$s, \$t	$\$d = \$s \times \$t$
div \$s, \$t	$\text{LO} = \$s \div \$t, \text{HI} = \$s \bmod \t
mflo \$d	$\$d = \text{LO}$ (thương)
mfhi \$d	$\$d = \text{HI}$ (phần dư)

Table 9.3: Các lệnh toán học

9.15 Ví dụ: Tính $\text{index} = \text{row} \times 7 + \text{col}$

```
# $t0 = row, $t1 = col
mul $t2, $t0, 7      # $t2 = row × 7
add $t2, $t2, $t1    # $t2 = row × 7 + col (index)

# Tính byte offset = index × 4
mul $t3, $t2, 4      # $t3 = index × 4
```

9.16 Memory Instructions

Chương trình sử dụng các lệnh truy cập bộ nhớ:

Lệnh	Mô tả
lw \$d, offset(\$s)	Load word: $\$d = \text{Memory}[\$s + \text{offset}]$
sw \$d, offset(\$s)	Store word: $\text{Memory}[\$s + \text{offset}] = \d
la \$d, label	Load address: $\$d = \text{địa chỉ của label}$
li \$d, imm	Load immediate: $\$d = \text{giá trị ngay lập tức}$

Table 9.4: Các lệnh truy cập bộ nhớ

9.17 Ví dụ: Đọc/ghi ô trên bàn cờ

```
# Đọc ô tại index
la $t0, boardP1      # Tải địa chỉ board
mul $t1, $t2, 4       # $t1 = index × 4 (byte offset)
add $t0, $t0, $t1     # $t0 = địa chỉ ô
lw $t3, 0($t0)        # Đọc giá trị ô vào $t3

# Ghi ô tại index
li $t3, 1             # Giá trị cần ghi
sw $t3, 0($t0)        # Ghi vào ô
```

10 Thiết kế giao diện người dùng

10.1 Tổng quan về giao diện

Giao diện người dùng (User Interface) là phần mà người chơi tương tác trực tiếp với chương trình. Giao diện tốt phải:

- Rõ ràng và dễ hiểu.
- Cung cấp feedback rõ ràng cho mỗi hành động.
- Hướng dẫn người chơi cách nhập liệu đúng.
- Tránh nhầm lẫn giữa các chế độ chơi.

10.2 Menu chính

Menu chính là màn hình đầu tiên người dùng thấy. Nó cho phép chọn giữa ba tùy chọn:

10.3 Hiển thị menu

```
===== BATTLESHIP GAME =====
```

1. Play Game
2. Instructions
3. Quit

Choose an option (1-3): _

10.4 Xử lý lựa chọn

```
IF choice == 1:  
    CALL choose_game_mode()  
    (Vào lựa chọn chế độ chơi)
```

```
IF choice == 2:  
    CALL show_help()  
    (Hiển thị hướng dẫn)
```

```
IF choice == 3:  
    CALL exit_program()  
    (Thoát)
```

10.5 Hướng dẫn (Help)

Hướng dẫn hiển thị luật chơi và cách nhập liệu:

```
===== BATTLESHIP GAME - INSTRUCTIONS =====
```

Objective: Destroy all opponent's ships.

Rules:

1. Board size: 7x7 (coordinates 0-6)



2. Each player has:

- 3 ships of length 2
- 2 ships of length 3
- 1 ship of length 4

Setup Phase:

- Place your ships on your board
- Each ship is defined by bow and stern coordinates:
Example: "0 0 0 1" means row_bow=0, col_bow=0, row_stern=0, col_stern=1
- Ship must be horizontal (same row) or vertical (same column)

Battle Phase:

- Take turns attacking opponent's board
- Enter target coordinates: "row col"
- HIT: You hit a ship
- MISS: You hit empty water
- First to destroy all opponent's ships wins!

Press any key to continue...

10.6 Lựa chọn chế độ chơi

Sau khi chọn Play Game, người chơi lựa chọn chế độ:

10.7 Hiện thị lựa chọn chế độ

===== CHOOSE GAME MODE =====

1. One Player (vs Computer)
2. Two Players

Choose a mode (1-2): _

10.8 Xử lý chế độ

IF mode == 1:

CALL setup_player(1)

CALL load_auto_board() (Tàu Computer tự động)



```
gameMode = 1

IF mode == 2:
    CALL setup_player(1)
    CALL setup_player(2)
    gameMode = 2

CALL game_loop()
```

10.9 Giai đoạn Setup

Trong giai đoạn setup, người chơi đặt tàu:

10.10 Hướng dẫn đặt tàu

```
===== PLAYER 1 - SETUP PHASE =====
Place your 6 ships on the 7x7 board.
```

Enter coordinates as: row_bow col_bow row_stern col_stern

Ship 1 (length 2): _

10.11 Phản hồi sau khi đặt tàu

Ship placed successfully!

Ship 2 (length 2): _

Hoặc nếu sai:

```
Invalid ship placement. Please try again.
Reason: Ship overlaps with existing ship.
```

Ship 1 (length 2): _

10.12 Hiện thị bàn cờ

Bàn cờ được in dưới dạng lưới 7×7 với tiêu đề hàng và cột:



10.13 Bàn cờ của chính mình

YOUR BOARD - Player 1

	0	1	2	3	4	5	6
0	[1]	[1]	[.]	[.]	[.]	[.]	[.]
1	[.]	[.]	[.]	[.]	[.]	[.]	[.]
2	[1]	[.]	[.]	[1]	[1]	[1]	[.]
3	[1]	[.]	[.]	[.]	[.]	[.]	[.]
4	[1]	[.]	[1]	[.]	[.]	[.]	[.]
5	[.]	[.]	[1]	[.]	[.]	[.]	[.]
6	[.]	[.]	[1]	[1]	[1]	[1]	[.]

Legend:

1 : Ship cell (still intact)

. : Empty cell

X : Destroyed ship cell

O : Missed shot (on own board)

10.14 Bản đồ bắn đối phương

ENEMY MAP - Player 2

	0	1	2	3	4	5	6
0	[.]	[X]	[.]	[.]	[.]	[.]	[.]
1	[O]	[.]	[.]	[.]	[.]	[.]	[.]
2	[.]	[.]	[.]	[.]	[.]	[.]	[.]
3	[.]	[.]	[.]	[.]	[.]	[.]	[.]
4	[.]	[.]	[X]	[.]	[.]	[.]	[.]
5	[.]	[.]	[.]	[.]	[.]	[.]	[.]
6	[O]	[.]	[.]	[.]	[.]	[.]	[.]

Legend:

. : Unknown cell (not attacked)

O : Miss

X : Hit

10.15 Dual-Board Display

Trong giai đoạn chiến đấu, hai bàn cờ được in song song để người chơi thấy cả hai cùng lúc:

YOUR BOARD - Player 1

```
  0 1 2 3 4 5 6
0 [1][1][.][.][.][.][.]
1 [.][][.][.][.][.][.]
2 [1][.][.][1][1][1][.]
3 [1][.][.][.][.][.][.]
4 [1][.][1][.][.][.][.]
5 [.][][1][.][.][.][.]
6 [.][][1][1][1][1][.]
```

ENEMY MAP - Player 2

```
  0 1 2 3 4 5 6
0 [.][][X][.][.][.][.]
1 [0][.][.][.][.][.][.]
2 [.][][.][.][.][.][.]
3 [.][][.][.][.][.][.]
4 [.][][X][.][.][.][.]
5 [.][][.][.][.][.][.]
6 [0][.][.][.][.][.][.]
```

10.16 Giai đoạn chiến đấu

Trong giai đoạn chiến đấu:

10.17 Yêu cầu nhập tọa độ

===== PLAYER 1 TURN =====

Enter target row and column (e.g., "2 3"): _

10.18 Thông báo kết quả

HIT!

(Cập nhật bàn cờ và bản đồ bắn)

Hoặc:

MISS!

(Cập nhật bản đồ bắn)

Hoặc nếu input sai:

You already attacked this cell! Try another.

Enter target row and column (e.g., "2 3"): _



10.19 Tạm dừng để đổi lượt (Pause for Swap)

Trong chế độ Two Players, sau khi mỗi lượt, chương trình tạm dừng để người chơi khác có cơ hội giảm bớt việc nhìn thấy bàn cờ của đối thủ:

```
===== PLEASE SWAP PLAYERS =====
```

```
Player 2, it's your turn now.
```

```
(Press any key to continue...)
```

Thủ tục pause_for_swap:

```
PROCEDURE pause_for_swap()
```

```
    PRINT message
```

```
    READ one character (to wait for user confirmation)
```

```
    (Optionally, clear screen to hide previous board)
```

```
END PROCEDURE
```

10.20 Thông báo kết thúc game

Khi một người chơi chiến thắng:

```
===== GAME OVER =====
```

```
PLAYER 1 WINS!
```

```
Congratulations! You destroyed all opponent's ships.
```

```
Return to menu? (Y/N): _
```

Hoặc:

```
===== GAME OVER =====
```

```
COMPUTER WINS!
```

```
Game Over. Better luck next time!
```

```
Return to menu? (Y/N): _
```

10.21 Thông báo lỗi

Chương trình cung cấp thông báo lỗi rõ ràng:

1. **Invalid input format:** Khi không thể tách số từ chuỗi input.
2. **Coordinates out of range:** Khi tọa độ ngoài $[0, 6]$.
3. **Ship overlaps with existing ship:** Khi đặt tàu chồng lấn.
4. **Ship must be horizontal or vertical:** Khi tàu chéo.
5. **Ship length does not match:** Khi độ dài sai.
6. **You already attacked this cell:** Khi bắn lại cùng ô.

10.22 Nguyên tắc thiết kế

Giao diện được thiết kế theo các nguyên tắc:

- **Clarity:** Rõ ràng về những gì người chơi cần làm tiếp theo.
- **Consistency:** Cùng format, cùng ký hiệu ở mọi nơi.
- **Feedback:** Phản hồi rõ ràng cho mỗi hành động.
- **Error Handling:** Thông báo lỗi cụ thể, không chung chung.
- **Accessibility:** Dễ sử dụng, không cần huấn luyện phức tạp.

11 Kế hoạch kiểm thử và các Test Case

11.1 Tổng quan về kiểm thử

Kiểm thử (Testing) là quá trình xác minh rằng chương trình hoạt động đúng theo yêu cầu. Kế hoạch kiểm thử bao gồm các test case được thiết kế để:

- Xác thực các chức năng chính hoạt động đúng.
- Phát hiện các trường hợp biên (boundary cases).
- Phát hiện các lỗi logic hoặc xử lý input.



11.2 Các loại kiểm thử

11.3 Kiểm thử giai đoạn Setup

Kiểm tra xem setup có đặt tàu đúng hay không.

11.4 Kiểm thử giai đoạn Battle

Kiểm tra xem cơ chế tấn công, phát hiện trúng/hạ, và kết thúc game hoạt động đúng hay không.

11.5 Kiểm thử xác thực input

Kiểm tra xem chương trình có từ chối input không hợp lệ hay không.

11.6 Kiểm thử tổng hợp

Kiểm tra toàn bộ quy trình từ menu → setup → battle → kết thúc.

11.7 Bảng Test Case

Dưới đây là bảng test case chi tiết:

Test Case	Input/Action	Expected Result	Purpose
TC-1	Menu: Choose 1 (Play Game)	Go to game mode selection	Verify main menu works
TC-2	Game Mode: Choose 1 (One Player)	Player 1 setup starts	Verify One Player mode selected
TC-3	Game Mode: Choose 2 (Two Players)	Player 1 setup starts	Verify Two Players mode selected
TC-4	Menu: Choose 2 (Instructions)	Display help text	Verify instructions display

Tiếp tục ở trang sau

Tiếp tục từ trang trước

TC-5	Menu: Choose 3 (Quit)	Program exits	Verify quit functionality
Setup Phase - Valid Ship Placement			
TC-6	Place horizontal ship: "0 0 0 1" (length 2)	Ship placed, display "Success"	Valid horizontal placement
TC-7	Place vertical ship: "2 3 4 3" (length 3)	Ship placed, display "Success"	Valid vertical placement
TC-8	Place ship: "6 4 6 6" (length 3)	Ship placed, display "Success"	Valid placement at boundary
TC-9	Place ship: "0 0 4 0" (length 5)	Ship placed, display "Success"	Valid long vertical ship
Setup Phase - Invalid Ship Placement			
TC-10	Place ship: "-1 0 -1 1" (out of range)	Reject, show error	Out of range coordinates
TC-11	Place ship: "7 0 7 1" (out of range)	Reject, show error	Out of range coordinates
TC-12	Place ship: "0 0 2 1" (diagonal/invalid)	Reject, show error	Invalid shape (not horizontal/vertical)
TC-13	Place ship: "0 0 0 3" (length 4 instead of 2)	Reject, show error	Length mismatch
TC-14	Place ship 1: "0 0 0 1", Ship 2: "0 0 0 1" (overlap)	Reject ship 2, show error	Overlapping ships

Tiếp tục ở trang sau



Tiếp tục từ trang trước

TC-15	Invalid input: "abc def ghi jkl" (non-numeric)	Reject, show error	Invalid input format
TC-16	Invalid input: "00" (only 2 numbers)	Reject, show error	Incomplete input
Battle Phase - Valid Attacks			
TC-17	Attack empty cell: "1 1" (board has no ship)	Display "MISS!", update memory	Valid miss
TC-18	Attack ship cell: "0 0" (board has ship there)	Display "HIT!", update board/memory	Valid hit
TC-19	Attack boundary: "6 6"	Depends on what's there	Valid boundary attack
Battle Phase - Invalid Attacks			
TC-20	Attack: "-1 0" (out of range)	Reject, show error	Out of range
TC-21	Attack: "7 5" (out of range)	Reject, show error	Out of range
TC-22	Attack same cell twice: "2 3", then "2 3"	Second attack rejected	Prevent repeated attacks
TC-23	Invalid input: "a b" (non-numeric)	Reject, show error	Invalid format
TC-24	Invalid input: "2" (only 1 number)	Reject, show error	Incomplete input

Tiếp tục ở trang sau

Tiếp tục từ trang trước

Game Flow			
TC-25	Complete setup for both players without errors	Both players' boards display with ships	Full setup phase passes
TC-26	Destroy all opponent's ships	Display "PLAYER X WINS!"	Game ends correctly
TC-27	Two Players mode: After Player 1 turn, display pause message	Display "Please swap players"	Turn swap works
TC-28	One Player mode: Computer attacks automatically	Computer selects valid cell, displays result	Computer AI works
TC-29	One Player mode: Destroy all Computer's ships	Display "PLAYER 1 WINS!"	Player can beat Computer
TC-30	One Player mode: All Player 1 ships destroyed	Display "COMPUTER WINS!"	Computer can beat Player
Data Integrity			
TC-31	After each successful placement, board[index]=1 for ship cells	Board updates correctly	Board state management

Tiếp tục ở trang sau

Tiếp tục từ trang trước

TC-32	After hit, opponent board[index]=0 at hit location	Opponent's ship marked destroyed	Board update on hit
TC-33	After miss, mem- ory[index]=1; board un- changed	Memory board updated, ship board un- changed	Board state sep- aration
TC-34	board_has_ship returns false when all cells=0	Game declares winner	Ship detection works

11.8 Chiến lược kiểm thử

11.9 Kiểm thử từ dưới lên (Bottom-up Testing)

Kiểm thử các thủ tục nhỏ trước:

1. parse_two_ints, parse_four_ints
2. board_has_ship
3. clear_board

Sau đó kiểm thử các thủ tục lớn:

1. read_and_place_ship
2. player_attack
3. game_loop

11.10 Kiểm thử biên (Boundary Testing)

Kiểm tra các giá trị tại biên:

- row/col = 0 (góc trên trái)



- row/col = 6 (góc dưới phải)
- Các ô biên khác

11.11 Kiểm thử tiêu cực (Negative Testing)

Kiểm tra input sai:

- Số âm
- Số quá lớn
- Ký tự chữ
- Để trống

11.12 Tiêu chí vượt qua kiểm thử

Chương trình được coi là vượt qua kiểm thử khi:

1. Tất cả test case phù hợp (TC-1 đến TC-34) cho kết quả như mong đợi.
2. Không có lỗi segmentation fault hoặc infinite loop.
3. Giao diện hiển thị rõ ràng, không có ký tự rác.
4. Tất cả thông báo lỗi hiển thị đúng.

11.13 Công cụ kiểm thử

Kiểm thử được thực hiện thông qua:

- **MARS Debugger**: Để theo dõi giá trị thanh ghi và bộ nhớ.
- **Manual Input**: Nhập liệu thủ công để kiểm tra.
- **Visual Inspection**: Kiểm tra output trên màn hình.



11.14 Báo cáo lỗi (Bug Report Format)

Nếu phát hiện lỗi, cần ghi lại:

Bug ID: [number]

Title: [brief description]

Steps to Reproduce:

1. [step 1]

2. [step 2]

...

Expected Behavior: [what should happen]

Actual Behavior: [what actually happened]

Severity: Critical / High / Medium / Low

Ví dụ:

Bug ID: 001

Title: Overlapping ships not detected in vertical placement

Steps to Reproduce:

1. Place Ship 1: "0 0 0 1" (horizontal, length 2)

2. Place Ship 2: "0 0 2 0" (vertical, overlaps at (0,0))

Expected Behavior: Ship 2 rejected with error message

Actual Behavior: Ship 2 accepted, overlaps with Ship 1

Severity: Critical

12 Giới hạn và cải tiến có thể

12.1 Giới hạn hiện tại

Chương trình Battleship trong phiên bản hiện tại có các giới hạn sau:

12.2 Chiến lược tấn công Computer

- **Không thông minh:** Computer không học hỏi từ các lần trúng trước. Nếu bắn trúng một ô, Computer sẽ không tập trung bắn vào các ô lân cận để tìm cả tàu.
- **Không ngẫu nhiên:** Chiến lược là xác định (deterministic), quét từ trái sang phải, từ trên xuống dưới. Người chơi có thể dự đoán được ô tiếp theo.

- **Tối ưu thấp:** Có thể tốn nhiều lượt bắn hơn cần thiết (lý thuyết tối ưu là bắn các ô cách nhau 2 ô, nhưng Computer bắn từng ô một).

12.3 Ghi lại lịch sử di chuyển (Move History)

- **Không lưu file:** Chương trình không ghi lại các di chuyển (moves) của cả hai người chơi vào file. Điều này có nghĩa là không thể xem lại lịch sử trò chơi sau khi kết thúc.
- **Mất dữ liệu:** Khi thoát chương trình, tất cả dữ liệu về các di chuyển sẽ mất.
- **Không hỗ trợ bonus:** Nếu assignment yêu cầu bonus cho tính năng ghi file, chương trình không đủ điều kiện.

12.4 Kích thước bàn cờ cố định

- **Chỉ hỗ trợ 7×7:** Bàn cờ là cố định 7×7. Không thể thay đổi thành 8×8, 10×10, hoặc kích thước khác.
- **Đội tàu cố định:** Số lượng và kích thước tàu là cố định (3 chiếc dài 2, 2 chiếc dài 3, 1 chiếc dài 4). Không thể tùy chỉnh.
- **Giảm tính linh hoạt:** Người dùng không thể cấu hình trò chơi theo sở thích.

12.5 Xử lý input hạn chế

- **Không xử lý số âm:** Input chứa số âm (ví dụ: “-1 2”) sẽ không được tách đúng. Parser chỉ xây dựng số dương.
- **Không hỗ trợ định dạng khác:** Nếu người dùng nhập định dạng khác (ví dụ: “row=2, col=3”), chương trình sẽ từ chối.
- **Tràn buffer:** Nếu input quá dài, có thể tràn inputBuffer và gây lỗi.

12.6 Giao diện text thuần

- **Không có clear screen:** Khi đổi lượt, các lệnh trước vẫn hiển thị trên màn hình. Người chơi khác có thể nhìn thấy.
- **Không tô màu:** Giao diện là text thuần, không có màu sắc để làm đẹp hoặc tăng sự tương tác.



- **Thiếu họa tiết:** Không có animation hay hiệu ứng trực quan nào.

12.7 Cải tiến có thể

Các cải tiến sau đây có thể được triển khai:

12.8 Computer AI nâng cao

- **Theo dõi các lần trúng:** Nếu bắn trúng ô (r, c) , Computer sẽ ưu tiên bắn các ô lân cận: $(r-1, c)$, $(r+1, c)$, $(r, c-1)$, $(r, c+1)$.
- **Xác định hướng tàu:** Khi bắn trúng ô thứ hai, Computer sẽ xác định tàu nằm ngang hay dọc, sau đó tiếp tục bắn theo hướng đó.
- **Ngẫu nhiên hóa:** Sử dụng random number generator để chọn ô ngẫu nhiên (thay vì quét từ trái sang phải).
- **Chiến lược parity:** Bắn vào các ô sao cho tất cả tàu đều bị phát hiện với số lượt bắn tối thiểu.

12.9 Ghi lại lịch sử trò chơi

- **Tạo file log:** Ghi tất cả các di chuyển vào một file text (ví dụ: `game_log.txt`).
- **Format log:** Mỗi dòng ghi: "Turn X: Player Y attacks (r, c) -> HIT/MISS"
- **Thống kê:** Tính số lượt bắn của mỗi người chơi, tỷ lệ trúng, v.v.
- **Replay:** Có thể tái hiện lại trò chơi từ file log.

12.10 Bàn cờ cấu hình được

- **Kích thước cấu hình:** Cho phép người dùng nhập kích thước bàn (ví dụ: 8×8 , 10×10).
- **Đội tàu cấu hình:** Cho phép định nghĩa số lượng tàu và kích thước của từng tàu.
- **Lưu cài đặt:** Lưu cài đặt yêu thích vào file để sử dụng lần sau.

12.11 Xử lý input tăng cường

- **Hỗ trợ số âm:** Parser phải theo dõi dấu âm.
- **Kiểm tra buffer overflow:** Xác thực độ dài input không vượt quá buffer size.
- **Hỗ trợ định dạng linh hoạt:** Chấp nhận nhiều định dạng (ví dụ: “0,0,0,1” hoặc “0 0 0 1”).

12.12 Giao diện cấp cao hơn

- **Clear screen:** Sử dụng lệnh hệ thống để xóa màn hình khi đổi lượt.
- **Tô màu:** Sử dụng ANSI color codes để in các ký hiệu với màu sắc (ví dụ: tàu màu xanh, trúng màu đỏ).
- **Animation:** Hiển thị animation khi bắn (ví dụ: hiệu ứng phát nổ khi trúng).
- **Sound:** Thêm âm thanh khi trúng/trượt (nếu MARS hỗ trợ).

12.13 Tính năng mạng

- **Chơi trực tuyến:** Cho phép hai người chơi ở máy tính khác nhau chơi với nhau qua mạng.
- **Server:** Cần một server để quản lý trò chơi và truyền thông tin.

Tuy nhiên, tính năng này vượt ra ngoài phạm vi assignment MIPS.

12.14 Thống kê và leaderboard

- **Theo dõi chiến tích:** Ghi lại số trò chơi thắng/thua của mỗi người chơi.
- **Leaderboard:** Hiển thị bảng xếp hạng các người chơi xuất sắc nhất.
- **Thống kê chi tiết:** Tỷ lệ thắng, số lượt bắn trung bình, v.v.

12.15 Mức ưu tiên cải tiến

Nếu cải tiến được triển khai, mức ưu tiên nên là:

Cải tiến	Độ khó	Ưu tiên
Computer AI nâng cao	Trung bình	Cao
Ghi lại lịch sử trò chơi	Trung bình	Cao (nếu có bonus)
Clear screen/tô màu	Thấp	Trung
Bàn cờ cấu hình được	Cao	Trung
Xử lý input tăng cường	Trung bình	Thấp
Tính năng mạng	Rất cao	Rất thấp
Thống kê/leaderboard	Cao	Thấp

Table 12.1: Mức ưu tiên cải tiến

12.16 Lý do không triển khai tính năng nâng cao

Các tính năng nâng cao không được triển khai trong phiên bản hiện tại vì:

1. **Phạm vi assignment:** Assignment tập trung vào luyện tập MIPS, không phải phát triển game hoàn chỉnh.
2. **Độ phức tạp:** Các tính năng như AI nâng cao hoặc tính năng mạng yêu cầu rất nhiều code.
3. **Giới hạn MARS:** MARS là simulator MIPS đơn giản, không hỗ trợ một số tính năng (ví dụ: socket, file I/O nâng cao).
4. **Thời gian:** Đủ thời gian để implement core functionality, không còn cho extras.

13 Kết luận

13.1 Tóm tắt chương trình

Chương trình Battleship được cài đặt thành công bằng ngôn ngữ lắp ráp MIPS trong môi trường MARS simulator. Chương trình mô phỏng đầy đủ trò chơi chiến lược cổ điển Battleship trên bàn cờ 7×7 với các tính năng chính sau:

13.2 Các tính năng được triển khai

1. **Giai đoạn Setup (Thiết lập):** Mỗi người chơi đặt 6 tàu (3 chiếc dài 2, 2 chiếc dài 3, 1 chiếc dài 4) lên bàn cờ 7×7 của mình. Chương trình kiểm tra tính hợp lệ đầy đủ: phạm vi, hình dạng (ngang/dọc), độ dài, không chồng lấn.
2. **Giai đoạn Battle (Chiến đấu):** Hai người chơi lần lượt tấn công bàn cờ đối phương bằng cách nhập tọa độ. Chương trình xác định trúng (HIT) hay trượt (MISS), cập nhật bàn cờ và bản đồ bắn tương ứng, ngăn chặn tấn công lặp lại.
3. **Kiểm tra kết thúc:** Sau mỗi lượt tấn công, chương trình kiểm tra xem bàn cờ của đối thủ còn tàu hay không. Khi tất cả tàu bị tiêu diệt, trò chơi kết thúc và thông báo người thắng.
4. **Menu và Hướng dẫn:** Chương trình cung cấp menu chính với các tùy chọn Play Game, Instructions, Quit, cùng với hướng dẫn rõ ràng về cách chơi.
5. **Hai chế độ chơi:**
 - **One Player:** Người chơi đấu với Computer. Computer tự động đặt tàu và tấn công theo chiến lược tuyến tính.
 - **Two Players:** Hai người chơi nhập liệu và tấn công lần lượt. Chương trình cung cấp điểm tạm dừng để đổi lượt bảo mật.
6. **Giao diện người dùng:** Hiển thị song song bàn cờ của mình và bản đồ bắn đối phương, giúp người chơi dễ dàng theo dõi tình hình trò chơi.

13.3 Kỹ năng MIPS được luyện tập

Thông qua assignment này, lập trình viên đã luyện tập các kỹ năng MIPS quan trọng:

1. **Arithmetic Instructions:** Sử dụng add, sub, mul, div để tính toán index mảng, độ dài tàu, và các giá trị số học.
2. **Memory/Data Transfer:** Sử dụng lw, sw, la, li để truy cập bộ nhớ, đặc biệt là các mảng 2D ánh xạ thành mảng 1D.
3. **Branch và Jump:** Sử dụng beq, bne, blt, bgt, j để điều khiển luồng chương trình, kiểm tra điều kiện, và tạo vòng lặp.

4. **Procedure Call:** Sử dụng jal và jr \$ra để gọi và trả về từ các thủ tục, hiện thực hóa lập trình có cấu trúc.
5. **Stack Usage:** Sử dụng stack để bảo toàn \$ra và \$s-registers khi các thủ tục gọi thủ tục khác.
6. **String/Integer Syscalls:** Sử dụng syscall để in chuỗi, in số, đọc chuỗi, đọc số, và đọc ký tự.
7. **Array Indexing:** Tính toán offset mảng từ index, xử lý các trường hợp phức tạp như mảng 2D.
8. **Conditional Logic:** Triển khai logic phức tạp để kiểm tra hợp lệ input, phát hiện trùng/hệt, xác định người thắng.

13.4 Thách thức và giải pháp

Trong quá trình cài đặt, có một số thách thức:

13.5 Thách thức: Quản lý bộ nhớ phức tạp

Với 4 bàn cờ (boardP1, boardP2, memP1, memP2), mỗi cái 49 từ, cần xác định đúng offset để tránh ghi đè dữ liệu.

Giải pháp: Sử dụng công thức rõ ràng: $\text{byte_offset} = (\text{row} \times 7 + \text{col}) \times 4$. Tất cả truy cập bộ nhớ tuân theo công thức này.

13.6 Thách thức: Bảo toàn thanh ghi khi thủ tục gọi thủ tục

Khi một thủ tục gọi thủ tục khác, \$ra sẽ bị ghi đè. Cần bảo toàn giá trị cũ của \$ra.

Giải pháp: Sử dụng stack. Tại đầu thủ tục, lưu \$ra vào stack trước khi gọi thủ tục khác. Tại cuối, khôi phục \$ra từ stack trước khi trả về.

13.7 Thách thức: Xác thực input phức tạp

Input cần được tách thành các số, kiểm tra phạm vi, kiểm tra hình dạng, độ dài, chồng lấn.

Giải pháp: Tách việc xác thực thành các bước riêng biệt, mỗi bước kiểm tra một điều kiện. Nếu bất kỳ kiểm tra nào thất bại, yêu cầu nhập lại.



13.8 Thách thức: Tách biệt boardP1/boardP2 và memP1/memP2

Dễ nhầm lẫn giữa board (trạng thái tàu) và memory (lịch sử tấn công). Viết vào board sai có thể phá hủy dữ liệu.

Giải pháp: Sử dụng những tên biến rõ ràng, viết comment chi tiết về mục đích của mỗi biến. Kiểm thử kỹ lưỡng.

13.9 Đóng góp cho việc học

Assignment này đóng góp đáng kể cho việc học lập trình MIPS:

1. **Thực hành toàn diện:** Không chỉ luyện tập một hoặc hai khái niệm, mà luyện tập hầu hết các khía cạnh của MIPS.
2. **Giải quyết vấn đề:** Cần suy luận để thiết kế data structures, thuật toán, và kiến trúc chương trình.
3. **Debugging:** Phát triển kỹ năng debug bằng cách sử dụng MARS debugger để theo dõi giá trị thanh ghi và bộ nhớ.
4. **Viết báo cáo kỹ thuật:** Thực hành viết báo cáo rõ ràng, không chèn code, nhưng giải thích thuật toán.

13.10 Kết luận cuối cùng

Chương trình Battleship MIPS đã được cài đặt thành công, thực hiện đầy đủ các yêu cầu của assignment. Chương trình:

OK Mô phỏng trò chơi Battleship đầy đủ trên bàn cờ 7×7 .

OK Hỗ trợ hai chế độ chơi (One Player vs Computer, Two Players).

OK Kiểm tra tính hợp lệ đầy đủ: phạm vi, hình dạng, độ dài, chồng lặn, tấn công lặp lại.

OK Phát hiện và thông báo trúng/hịt đúng đắn.

OK Xác định người thắng chính xác.

OK Cung cấp giao diện người dùng rõ ràng, dễ sử dụng.



OK Sử dụng hiệu quả các khái niệm MIPS: arithmetic, memory, branch, procedure, stack, syscall.

Assignment này không chỉ đạt yêu cầu đề bài mà còn là một ứng dụng đủ đầy, hoạt động tốt và có giá trị luyện tập cao. Nó là một ví dụ tốt về cách cài đặt một chương trình có độ phức tạp trung bình bằng ngôn ngữ lắp ráp, đòi hỏi kỹ năng lập trình, thiết kế data structures, và khả năng debugging.

13.11 Hướng phát triển trong tương lai

Như đã đề cập trong chương Giới hạn và Cải tiến, những hướng phát triển tiếp theo có thể bao gồm:

- Cải tiến AI cho Computer (chiến lược thông minh, ngẫu nhiên hóa, tối ưu bản).
- Ghi lại lịch sử trò chơi vào file (move logging, replay).
- Hỗ trợ bàn cờ và đội tàu cấu hình được.
- Giao diện cấp cao hơn (tô màu, animation, clear screen).
- Thống kê và leaderboard.

Tuy nhiên, các tính năng này vượt ra ngoài phạm vi của assignment MIPS lần này.

Để kết thúc, assignment Battleship MIPS là một dự án lập trình hữu ích, có độ khó vừa phải, giúp lập trình viên làm quen với các khái niệm lắp ráp, cấu trúc dữ liệu, và lập trình có cấu trúc trong MIPS.